

EE798R: Intelligent Pattern Recognition

Course Project Report

Lightweight Multimodal Document Retrieval with ColSmolVLM

Model Scaling, Late Interaction, and Token Compression for Visual RAG

Pratyush Singh

Roll number: 230784

Project scope. This course project studies the retrieval stage of a multimodal retrieval-augmented generation (RAG) system. It evaluates released ColSmolVLM checkpoints; it does not claim to train the models or evaluate an answer generator.

September 2026

Abstract

A retrieval-augmented generation system can answer from a document only after it finds the page containing the evidence. This is difficult for visually rich PDFs because meaning may be carried by a table layout, a plot, a logo, or the position of a small label—not only by plain text. This course project studies that retrieval problem step by step. It compares two OCR-based text methods, a visual method that reduces a whole page to one vector, and a visual method that keeps many local page vectors and matches them using MaxSim. The experiments use the complete DocVQA, InfoVQA, ArxivQA, and TatDQA retrieval test sets. The 500M MaxSim model obtains nDCG@5 scores of 57.8, 86.9, 74.7, and 76.5, respectively, and is the best tested method on every dataset. The project also studies efficiency: k-means compression keeps at least 95% of the original retrieval quality while storing only 12.5–25% of the page tokens. The report explains the underlying mathematics with small examples, connects each equation to the implementation, and discusses where the system succeeds, where it fails, and what would still be needed to turn the retriever into a complete multimodal RAG application.

1 Why this project is needed

Consider a student who uploads hundreds of lecture slides, research papers, and scanned forms, then asks: “What does the colour gradient in this scientific figure represent?” A language model may produce fluent text, but it has two problems. First, the required fact may not be stored in its parameters. Second, even if the model gives a plausible answer, it may not be grounded in the student’s document.

RAG addresses this by treating the document collection as an external memory [1]. A complete RAG system performs two main operations:

question $\longrightarrow$ retrieve evidence $\longrightarrow$ generate an answer from that evidence.

The complete relationship can be written as

$$\hat{a} = G(q, \text{TopK}_{D \in \mathcal{C}} S(q, D)), \quad (1)$$

where:

- q is the user’s question;
- $\mathcal{C}$ is the collection of document pages;
- $S(q, D)$ measures how relevant page D is to question q ;
- TopK keeps the K highest-scoring pages;
- G is a generator, and $\hat{a}$ is its final answer.

The important point is not the notation. It is the order of events: *retrieval happens before generation*. If the correct page is not among the retrieved pages, the generator does not receive the evidence. This project therefore concentrates on the retrieval score $S(q, D)$.

1.1 Why ordinary text retrieval can fail on PDFs

The usual document pipeline is

PDF $\rightarrow$ OCR $\rightarrow$ text chunks $\rightarrow$ embeddings $\rightarrow$ nearest neighbours.

It is practical, but OCR turns a two-dimensional page into a one-dimensional string. A table’s row–column relationship, a plot legend, small corner text, and typographic hierarchy may be lost or reordered. The problem is cumulative:

OCR error $\rightarrow$ incomplete text $\rightarrow$ weaker embedding $\rightarrow$ wrong page retrieved.

A text encoder cannot recover visual information that was removed before the encoder saw the page.

This project studies the alternative:

page image $\rightarrow$ visual token embeddings $\rightarrow$ late interaction $\rightarrow$ relevant pages.

The page image remains intact. The model can therefore use words, layout, graphics, and spatial relationships together. The generator is intentionally outside the experimental boundary: the goal of the course project is to understand the representation and matching stages that would supply a future generator with evidence.

1.2 What the project tries to learn

The work is organised around four simple questions.

1. Does direct visual retrieval beat OCR-based lexical and dense-text baselines?
2. Is a set of local page vectors better than one globally pooled vector?
3. Does the larger 500M ColSmolVLM checkpoint justify its extra size over 256M?
4. How many page tokens can be removed before retrieval quality collapses?

2 Background: how the main ideas connect

This section places the project in context. Each earlier method solves one part of the problem; the tested ColSmolVLM pipeline combines their useful ideas.

RAG. Lewis et al. combine a parametric sequence model with retrieved external memory [1]. External documents are easier to update and inspect than facts hidden in model parameters. This project isolates the retrieval term $S(q, D)$ in Equation 1; answer generation is not measured.

Text retrieval. BM25 is a strong sparse baseline that rewards exact term overlap while normalising for document length [2]. Dense bi-encoders instead map a query and a document to semantic vectors and compare them by cosine similarity. We use BGE-small-en-v1.5 as a compact dense baseline [3].

Late interaction. ColBERT encodes query and document tokens independently, then performs a fine-grained MaxSim comparison at retrieval time [4]. This preserves local evidence while allowing document vectors to be computed offline.

Visual document retrieval. ColPali transfers late interaction to rendered document pages: a vision-language model emits multiple page vectors directly from the image, avoiding an OCR-first representation bottleneck [5]. The same work introduces ViDoRe, a benchmark spanning visually rich page-retrieval tasks. ColSmolVLM applies the idea to compact SmolVLM backbones; the released checkpoints are designed for multi-vector retrieval [6, 7].

3 Understanding the retrieval mathematics

This section builds the mathematics gradually. Every retrieval method performs the same final task—assign a score to each page—but it represents the page differently.

3.1 Step 1: ranking the pages

Suppose the collection contains N pages,

$$\mathcal{C} = \{D_1, D_2, \dots, D_N\}. \quad (2)$$

For one question q , the system calculates one score per page:

$$s_j = S(q, D_j), \quad j = 1, 2, \dots, N. \quad (3)$$

The pages are then sorted from the largest score to the smallest. If $\pi(1)$ is the index of the highest-scoring page, the returned order satisfies

$$S(q, D_{\pi(1)}) \geq S(q, D_{\pi(2)}) \geq \dots \geq S(q, D_{\pi(N)}). \quad (4)$$

For example, scores $[0.20, 0.85, 0.40]$ produce the page order $[D_2, D_3, D_1]$. Retrieval succeeds if a relevant page appears near the beginning. The pattern-recognition decision therefore depends on two choices:

1. how the question and page are represented; and
2. how those representations are compared to produce $S(q, D)$.

3.2 Step 2: text-based comparison

BM25: matching important words

BM25 begins with exact terms. A term is useful when it occurs in the page but is uncommon across the whole collection. Its inverse-document-frequency weight is

$$\text{IDF}(t) = \log \left(\frac{N - n_t + 0.5}{n_t + 0.5} + 1 \right), \quad (5)$$

where N is the number of pages and n_t is the number containing term t . A rare term such as a product code receives more weight than a common term such as “the”.

BM25 then combines these term weights:

$$\text{BM25}(q, D) = \sum_{t \in q} \text{IDF}(t) \times \underbrace{\frac{f(t, D)(k_1 + 1)}{f(t, D) + k_1 \left(1 - b + b \frac{|D|}{|\overline{D}|} \right)}}_{\text{term-frequency and length adjustment}}. \quad (6)$$

Here $f(t, D)$ counts the term in the OCR text, $|D|$ is that text’s length, and $|\overline{D}|$ is average length. We use $k_1 = 1.5$ and $b = 0.75$. The fraction increases when the term repeats, but it saturates instead of growing without limit. It also prevents long pages from winning merely because they contain more words. BM25 is excellent for exact names and numbers, but it cannot match information that OCR did not recognise.

Dense text retrieval: matching meaning

The BGE baseline converts the question and the OCR text into vectors $\mathbf{e}_q$ and $\mathbf{e}_D$. Their cosine similarity is

$$S_{\text{text}}(q, D) = \frac{\mathbf{e}_q^\top \mathbf{e}_D}{\|\mathbf{e}_q\|_2 \|\mathbf{e}_D\|_2}. \quad (7)$$

The numerator is a dot product. The denominator removes vector length, leaving only direction. A score near 1 means the vectors point in similar directions; near 0 means they are unrelated. Unlike BM25, this can connect paraphrases such as “yearly income” and “annual revenue”. It still only sees the extracted OCR text.

3.3 Step 3: representing the page as an image

A single global visual vector

The visual encoder divides a page into many learned regions or tokens. It returns

$$D = [\mathbf{d}_1, \mathbf{d}_2, \dots, \mathbf{d}_n], \quad \mathbf{d}_j \in \mathbb{R}^d, \quad (8)$$

where n is the number of page tokens and $d = 128$ is the embedding size in this project. The simplest way to obtain one page vector is to average them:

$$\bar{\mathbf{d}} = \frac{1}{n} \sum_{j=1}^n \mathbf{d}_j. \quad (9)$$

The question tokens are averaged in the same way, producing $\bar{\mathbf{q}}$. The global score is then

$$S_{\text{global}}(q, D) = \bar{\mathbf{q}}^\top \bar{\mathbf{d}}. \quad (10)$$

This is compact: every page becomes only one vector. However, a small chart label is averaged together with headings, body text, white space, and every other page region. A rare but decisive local signal can disappear inside the mean.

Keeping local vectors: MaxSim late interaction

ColSmolVLM avoids early averaging. It keeps m question vectors

$$Q = [\mathbf{q}_1, \mathbf{q}_2, \dots, \mathbf{q}_m] \quad (11)$$

and n page vectors

$$D = [\mathbf{d}_1, \mathbf{d}_2, \dots, \mathbf{d}_n]. \quad (12)$$

All vectors are L_2 -normalised, so $\mathbf{q}_i^\top \mathbf{d}_j$ is their cosine similarity. Scoring now happens in three understandable operations:

1. compare every question token i with every page token j ;
2. for each question token, keep only its best page match;
3. add those best-match values.

This is the MaxSim equation:

$$S_{\text{MaxSim}}(Q, D) = \sum_{i=1}^m \max_{1 \leq j \leq n} \mathbf{q}_i^\top \mathbf{d}_j. \quad (13)$$

The inner maximum lets each query concept choose a different location on the page. The outer sum requires the page to match many parts of the question, not just one lucky word.

Worked example. Imagine three important query tokens—“net”, “income”, and “2019”—and four page regions. Computing all 3×4 dot products gives

$$QD^\top = \begin{bmatrix} 0.92 & 0.10 & 0.08 & 0.21 \\ 0.12 & 0.88 & 0.14 & 0.18 \\ 0.07 & 0.19 & 0.91 & 0.15 \end{bmatrix}. \quad (14)$$

Read one row at a time:

$$\text{best match for “net”} = \max(0.92, 0.10, 0.08, 0.21) = 0.92, \quad (15)$$

$$\text{best match for “income”} = \max(0.12, 0.88, 0.14, 0.18) = 0.88, \quad (16)$$

$$\text{best match for “2019”} = \max(0.07, 0.19, 0.91, 0.15) = 0.91. \quad (17)$$

Therefore,

$$S_{\text{MaxSim}} = 0.92 + 0.88 + 0.91 = 2.71. \quad (18)$$

The three strongest matches occur in three different page regions, which is exactly what a financial table may require. Early global averaging can blur these separate cells; MaxSim keeps them available until the question arrives.

What MaxSim costs

Every one of the m question vectors is compared with n page vectors, and each dot product uses d numbers. The work per question–page pair therefore grows as

$$\text{operations} \propto m \times n \times d. \quad (19)$$

The page vectors are stored before questions arrive. With N pages and FP16 values (2 bytes per number), approximate index memory is

$$M_{\text{index}} = N n d \times 2 \text{ bytes}. \quad (20)$$

For a simple example, 500 pages with 1,000 tokens/page and $d = 128$ require

$$500 \times 1000 \times 128 \times 2 = 128,000,000 \text{ bytes} \approx 122 \text{ MiB}. \quad (21)$$

This is why token compression is useful: reducing n reduces both stored values and comparisons.

3.4 Step 4: reducing the number of page tokens

Let r be the fraction of tokens to retain. A page with n tokens is reduced to

$$k = \max(1, \lceil rn \rceil). \quad (22)$$

For example, $n = 1,000$ and $r = 0.25$ gives $k = 250$. We test $r \in \{1, .75, .50, .25, .125\}$ with four strategies:

- **random:** sample k tokens without replacement (three seeds);
- **uniform:** retain approximately evenly spaced token positions;
- **mean-pool:** divide the sequence into k contiguous groups and average each group;
- **k-means:** replace the tokens with k learned centroids.

The first two choose existing tokens. Mean-pooling and k-means instead construct representative tokens. K-means chooses centroids $\mathbf{c}_1, \dots, \mathbf{c}_k$ that approximately minimise

$$\min_{\{\mathbf{c}_\ell\}_{\ell=1}^k} \sum_{j=1}^n \min_{\ell} \|\mathbf{d}_j - \mathbf{c}_\ell\|_2^2, \quad (23)$$

which means: assign each original page token to its nearest centroid, then make the total squared distance as small as possible. In plain language, the centroids summarise groups of similar page regions.

3.5 Step 5: measuring whether the ranking is good

Recall@ K and reciprocal rank

Recall@ K asks a yes/no question for each query: did at least one relevant page appear in the first K results? The reported value is the fraction of queries for which the answer is yes. Reciprocal rank also cares about the first relevant position r_q :

$$\text{RR}(q) = \frac{1}{r_q}, \quad \text{MRR} = \frac{1}{|Q|} \sum_{q \in Q} \text{RR}(q). \quad (24)$$

A relevant page at rank 1 gives 1, rank 2 gives 0.5, and rank 10 gives 0.1.

nDCG@ K : rewarding early relevant pages

The main metric is nDCG@5. First, discounted cumulative gain assigns less credit to relevant pages found later:

$$\text{DCG@}K = \sum_{i=1}^K \frac{2^{g_i} - 1}{\log_2(i + 1)}. \quad (25)$$

Here g_i is the relevance gain at rank i . The logarithm is a discount: the denominator grows as rank becomes worse. IDCG@ K is the best possible DCG for the same relevance labels. Normalisation gives

$$\text{nDCG@}K = \frac{\text{DCG@}K}{\text{IDCG@}K}, \quad 0 \leq \text{nDCG@}K \leq 1. \quad (26)$$

For a single binary-relevant page, retrieving it at rank 1 gives nDCG= 1. Retrieving it at rank 3 gives

$$\text{nDCG@}3 = \frac{1/\log_2(4)}{1/\log_2(2)} = \frac{1}{2} = 0.5. \quad (27)$$

Thus nDCG captures the behaviour desired in RAG: evidence should not merely be present somewhere; it should appear near the top of the context supplied to the generator.

4 How the project was carried out

4.1 Datasets

We use the complete public test splits exposed through MTEB/ViDoRe, with no locally supplied PDFs. Table 1 shows the exact evaluation sizes. Each item is a natural-language query with page-level relevance labels.

Table 1: Evaluation collections and usable OCR coverage. Mapping coverage was 100% for all four datasets; “usable OCR” counts pages with non-empty extracted text.

Dataset	Queries	Pages	Usable OCR	Dominant visual content
DocVQA	451	500	496 (99.2%)	scanned forms and documents
InfoVQA	494	500	497 (99.4%)	infographics
ArxivQA	500	500	422 (84.4%)	scientific plots and figures
TatDQA	1,646	277	277 (100%)	financial tables and reports

The OCR pages are joined to the MTEB corpus by exact image digest (or filename when available). The code distinguishes a mapped-but-empty OCR row from a missing mapping. This matters most

for ArxivQA: its text scores are valid for the supplied OCR artifact, but 78 pages contain no OCR text, so visual–text comparisons there should be interpreted cautiously.

4.2 Systems compared in the project

Table 2: Experimental factors. All released neural checkpoints are evaluated without project-specific fine-tuning.

Family	Representation	Score	Purpose
BM25	Tesseract OCR term counts	BM25	lexical text baseline
BGE-small	one 384-D OCR-text vector	cosine	semantic text baseline
Global visual	one pooled 128-D page vector	dot product	visual single-vector ablation
ColSmol MaxSim	many 128-D page vectors	Eq. 13	proposed retrieval mechanism

The visual experiments compare `vidore/colSmol-256M` and `vidore/colSmol-500M`. The 500M checkpoint contains 477,663,552 parameters in the loaded implementation. All neural inference is frozen; no labels from the evaluation sets are used for training.

4.3 End-to-end retrieval procedure

The same evaluation procedure is used for every visual model and dataset:

1. load the page images, questions, and page-level relevance labels;
2. encode every page once and save its visual token matrix D ;
3. encode one incoming question to obtain its token matrix Q ;
4. compute either the global score or the MaxSim score for every page;
5. sort all pages by decreasing score and record the relevant page’s rank;
6. repeat for every question, then calculate nDCG@5, MRR, and Recall@ K ;
7. for the compression experiment, compress each stored D before Step 4 and repeat the evaluation;
8. save aggregate metrics, per-question ranks, timings, memory use, and provenance to JSON.

Each query is compared against the complete corpus for its dataset. Therefore, a score improvement cannot come from evaluating an easier subset of pages.

4.4 Implementation and reproducible execution

Experiments ran with seed 42 in Python 3.12.12, PyTorch 2.11.0+cu130, Transformers 5.17.0, and ColPali Engine 0.3.18 on one NVIDIA RTX A6000 (physical GPU 2, 47.4 GiB). FP16 is used for visual embeddings. Dataset fingerprints, checkpoint revisions, timings, per-query ranks, and environment information are stored with the results. Twenty automated unit tests passed in the final server run.

The implementation follows the mathematical structure directly:

Listing 1: Core late-interaction scorer (simplified from the project implementation).

```
# Q: [batch, query_tokens, dim]
# D: [page_tokens, dim]
similarities = Q @ D.T
score = similarities.max(dim=-1).values.sum(dim=-1)
```

The full experiment driver evaluates both checkpoints, all four datasets, two text baselines, 100 compression configurations, and then regenerates summary tables, plots, and failure cases. Existing valid caches are reused to make interrupted runs resumable.

5 What the experiments showed

5.1 Main comparison: which method retrieves best?

Table 3: nDCG@5 (%). Best result per dataset is bold. “Global” and “MaxSim” use page images; BM25 and BGE use OCR text.

Representation	Model	DocVQA	InfoVQA	ArxivQA	TatDQA
Text lexical	BM25	34.8	60.9	18.5	57.4
Text dense	BGE-small-en-v1.5	27.1	68.5	32.3	31.4
Global visual	ColSmol-256M	16.6	45.4	18.3	21.3
Global visual	ColSmol-500M	19.1	55.5	25.3	28.4
MaxSim visual	ColSmol-256M	55.6	83.6	72.4	75.7
MaxSim visual	ColSmol-500M	57.8	86.9	74.7	76.5

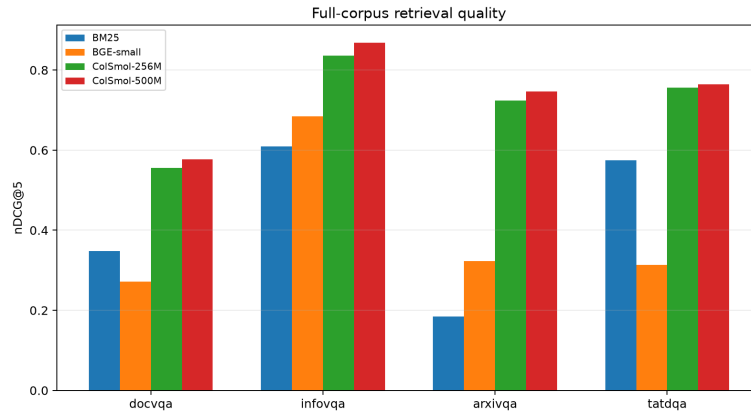


Figure 1: Main nDCG@5 comparison generated from the committed result JSON files.

Three observations tell the main story.

1. **Late interaction is the decisive component.** With the same 500M encoder, MaxSim exceeds global pooling by 38.6 points on DocVQA, 31.4 on InfoVQA, 49.4 on ArxivQA, and 48.1 on TatDQA. Local matching is therefore not a cosmetic scoring change; it prevents crucial page regions from being averaged away.
2. **Visual retrieval is consistently strongest.** MaxSim-500M beats the best text baseline by 23.0, 18.4, 42.4, and 19.0 points on the four datasets. The largest gap occurs on scientific figures, exactly where OCR has both a representational disadvantage and incomplete text coverage.
3. **Scaling helps, but less than representation.** Moving from 256M to 500M improves MaxSim by 2.2, 3.2, 2.2, and 0.8 points. By contrast, changing global pooling to MaxSim gives tens of points. For this scale, keeping local tokens is more important than doubling the checkpoint class.

For the 500M MaxSim system, Recall@1 / Recall@5 / Recall@10 is 50.0/64.3/71.5 on DocVQA, 82.4/90.5/92.7 on InfoVQA, 67.0/81.2/85.8 on ArxivQA, and 64.1/87.4/93.7 on TatDQA. Thus,

most remaining errors are not only ordering errors: on DocVQA, 28.5% of queries still have no relevant page in the top ten.

5.2 Compression experiment: how many tokens are necessary?

Table 4: Smallest configuration retaining at least 95% of the uncompressed nDCG@5. All selected configurations use k-means.

Dataset	Tokens kept	Base nDCG	Compressed nDCG	Base MB	Compressed MB
DocVQA	25%	57.8	56.6	129.36	32.36
InfoVQA	12.5%	86.9	83.4	87.08	10.84
ArxivQA	12.5%	74.7	71.8	107.02	13.35
TatDQA	25%	76.5	73.0	72.38	18.11

Equation 20 predicts the almost linear storage reduction visible in Table 4: retaining one eighth of the tokens gives roughly one eighth of the index. K-means is the strongest aggressive compressor because its centroids preserve occupied regions of embedding space. Uniform and random selection may discard rare but decisive tokens; contiguous mean pooling may mix unrelated neighbouring regions.

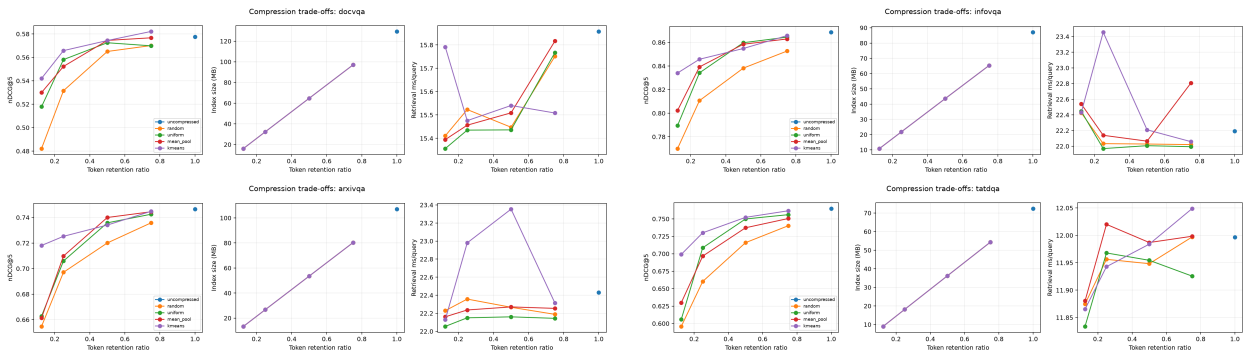


Figure 2: Quality-storage trade-offs for four compression methods. Random results aggregate three seeds; deterministic methods use one run.

Compression did *not* consistently reduce measured query latency. For example, DocVQA falls from 16.06 to 15.48 ms/query at its chosen point, whereas InfoVQA rises from 16.68 to 22.43 ms/query. At these corpus sizes, GPU launch, batching, padding, top- k , and sorting overhead can dominate the number of dot products. The defensible conclusion is therefore strong memory savings with preserved quality, not a universal latency speed-up.

5.3 Computational requirements

Table 5: Measured 500M visual-system efficiency. Encoding peak VRAM is about 2.9 GiB on every dataset.

Dataset	Page encode (ms)	Query encode (ms)	MaxSim (ms/query)	Index (MB)
DocVQA	693.1	16.5	16.1	129.36
InfoVQA	500.6	16.4	16.7	87.08
ArxivQA	577.7	21.2	22.7	107.02
TatDQA	706.2	16.5	9.1	72.38

Page encoding is the expensive offline operation; query encoding and scoring are online. The index remains small here because the benchmark corpora contain only 277–500 pages. For a million pages with 1,000 tokens/page, $d = 128$, and FP16 storage, Equation 20 gives approximately 238 GiB, which motivates compression or specialised multi-vector indexing for deployment.

6 Looking at individual successes and failures

Table 6 traces individual questions through the rankings. These are not generated explanations of page content; they are direct case studies from the stored per-query ranks.

Table 6: Representative retrieval cases. Lower rank is better.

Dataset	Query	MaxSim	Global	BM25	BGE
ArxivQA	What can be inferred about the term “alexa” in the given figure?	1	234	400	478
InfoVQA	What does “R” stand for?	1	239	293	430
TatDQA	What was the percentage change in the net total between 2018 and 2019?	1	243	137	177
InfoVQA	Which is element 67 on the periodic table?	44	23	1	1
DocVQA	What are the letters in the monogram logo at the top right?	447	497	45	339
ArxivQA	What does the figure primarily illustrate?	385	286	480	4

The first three cases show the advantage of local visual evidence. A term inside a plot, a one-letter infographic label, or a relationship spanning table cells can be matched without requiring OCR to linearise the whole page. The periodic-table case shows the opposite: a distinctive fact expressed cleanly in text is ideal for lexical or dense text retrieval.

The failures are equally informative. A tiny monogram in a named corner requires both fine visual resolution and explicit spatial grounding; neither representation reliably solves it. The generic question “What does the figure primarily illustrate?” contains few discriminative words, so many scientific pages can look semantically plausible. BGE ranks the relevant page fourth, demonstrating that visual and text channels make complementary errors rather than forming a strict hierarchy.

7 What we learned and what remains limited

What was learned. The strongest conclusion is architectural: preserving a set of local descriptors and delaying interaction until the query arrives is dramatically more effective than compressing a page to one vector. The 500M model is the best tested system, but the 256M MaxSim model already beats every text and global baseline. K-means then recovers much of the storage cost without erasing the advantage.

OCR comparability. Text baselines depend on supplied Tesseract artifacts. Non-empty OCR coverage is 84.4% on ArxivQA, while the visual methods see all 500 page images. This is a real operational advantage of the visual route, but it also means that the ArxivQA gap cannot be attributed purely to representation quality. The other three datasets have 99.2–100% non-empty OCR coverage.

Scale. Full benchmark splits were used, but each retrieval corpus is small. Brute-force GPU MaxSim is reasonable for 277–500 pages and does not prove million-page scalability. Approximate

multi-vector search, sharding, candidate generation, or reranking would be necessary at production scale.

No task-specific training. This is an evaluation and ablation project. It uses released ColSmol checkpoints and does not fine-tune on the four test sets. That choice gives a clean study of off-the-shelf representations, but does not answer how contrastive fine-tuning or parameter-efficient adaptation would change the trade-off.

No answer-generation evaluation. Although RAG motivates the work, the experiments stop after retrieval. There is no language-model answer, faithfulness metric, or hallucination study. Calling the current system a complete RAG pipeline would therefore be incorrect. The honest claim is that it implements and evaluates the retriever required by a future visual RAG system.

Qualitative evidence. Per-query successes and failures are analysed, satisfying the need for qualitative interpretation, but no token-to-region heatmaps were generated. Future work should overlay each query token’s argmax location from Equation 13 on the page. This would turn the ranking score into a spatial explanation and help distinguish encoder failures from ambiguous annotations.

8 From this retriever to a complete RAG system

The next engineering step is short, but its evaluation is a separate experiment:

1. index each page once with compressed ColSmol vectors;
2. retrieve the top- K page images with MaxSim;
3. give those images and the question to a vision-language generator;
4. require citations to page identifiers;
5. score retrieval and generation separately, including answer correctness and faithfulness.

This separation is important. Retrieval Recall@ K asks whether the evidence was available; answer accuracy asks whether the generator used it; faithfulness asks whether the answer is actually supported. Conflating the three makes a failure impossible to diagnose.

9 Conclusion

This project began with a simple fact: generation cannot use evidence that retrieval fails to find. For visually rich documents, OCR-first text retrieval discards part of the signal before matching begins. ColSmolVLM instead represents a page as many local visual vectors, and MaxSim lets each query concept find its strongest region. On four complete ViDoRe tasks, that design beats BM25, dense OCR-text retrieval, and global visual pooling. The largest gain comes not from model size but from retaining local structure. K-means compression then keeps at least 95% of baseline nDCG@5 with only one eighth to one quarter of the visual tokens. The resulting system is a lightweight, reproducible, and mathematically interpretable retriever—a strong foundation for multimodal RAG, while remaining explicit about what has not yet been generated or evaluated.

Implementation provenance and academic integrity

The ColSmolVLM checkpoints, ColPali/ColBERT concepts, BGE checkpoint, MTEB/ViDoRe datasets, and Tesseract OCR artifacts are prior work and are cited below. The project contri-

bution is the reproducible evaluation implementation, verified OCR-to-corpus join, global-pooling ablation, 256M/500M comparison, token-compression study, efficiency measurement, aggregation, plots, and per-query failure analysis. No benchmark result is presented as model training performed in this project.

References

- [1] P. Lewis et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” *NeurIPS*, 2020. <https://arxiv.org/abs/2005.11401>
- [2] S. Robertson and H. Zaragoza, “The Probabilistic Relevance Framework: BM25 and Beyond,” *Foundations and Trends in Information Retrieval*, 3(4), 2009. <https://doi.org/10.1561/15000000019>
- [3] S. Xiao et al., “C-Pack: Packaged Resources To Advance General Chinese Embedding,” 2023. <https://arxiv.org/abs/2309.07597>
- [4] O. Khattab and M. Zaharia, “ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT,” *SIGIR*, 2020. <https://doi.org/10.1145/3397271.3401075>
- [5] M. Faysse et al., “ColPali: Efficient Document Retrieval with Vision Language Models,” *ICLR*, 2025. <https://arxiv.org/abs/2407.01449>
- [6] ViDoRe, “colSmol-500M Model Card.” <https://huggingface.co/vidore/colSmol-500M>
- [7] A. Marafioti et al., “SmolVLM Grows Smaller—Introducing the 256M & 500M Models,” Hugging Face, 2025. <https://huggingface.co/blog/smolervlm>
- [8] ViDoRe, “Visual Document Retrieval Benchmark Collection.” <https://huggingface.co/collections/vidore/vidore-benchmark>

A Artifact map

The report can be audited from repository artifacts rather than reconstructed from prose:

- `artifacts/results/visual/`: eight full visual evaluations;
- `artifacts/results/text/`: four OCR text-baseline evaluations;
- `artifacts/results/compression/`: 100 compression configurations and manifests;
- `artifacts/summary/main_results.csv`: 24 primary method–dataset rows;
- `artifacts/summary/compression_aggregate.csv`: 68 aggregated compression rows;
- `artifacts/summary/failure_analysis.csv`: 30 difficult queries;
- `artifacts/results/environment_diagnostic.json`: software and GPU provenance.

B Minimal reproduction command

The repository README documents every experiment individually. The complete tested pipeline is:

```
source rag/bin/activate
GPU_ID=2 bash scripts/run_all_experiments.sh
```